UADER FCyT

IDTILAB
Laboratorio de Investigación y Desarrollo en Tecnologías Informáticas

Trabajo Final Integrador - Museo de Redes Neuronales

Licenciatura en Sistemas de Información

Cátedra: Inteligencia Artificial

Integrantes:

Mauro Mottet: Líder y responsable del código y repositorio
Agustín Scheffler: Responsable de arquitectura
Candela Rojas: Responsable de documentación
Florencia Colazo: Responsable de test

Equipo docente:

Dra. Daniela López De Luise
Alejandro Pérez

Fecha: 19/10/2026

Manual de Referencia:

1. Introducción

CarGross.py es un programa en Python que implementa la red neuronal de Carpenter/Grossberg, también conocida como ART-1, siguiendo el algoritmo que describe Lippmann (1987) en el Box 3 de su artículo.
La red agrupa patrones binarios sin necesidad de etiquetas previas; por eso su aprendizaje es no supervisado. Recibe los patrones de a uno y, para cada uno, busca si se parece lo suficiente a alguno de los prototipos aprendidos. Si lo encuentra, lo asigna a ese clúster; si no, crea una categoría nueva. El grado de parecido exigido lo fija el parámetro de vigilancia ρ (rho), un número entre 0 y 1: cuanto más alto, más estricta es la red.
En este trabajo la red se aplica a la telemetría de un servidor, es decir, a mediciones periódicas de uso de CPU, memoria, disco y red, para descubrir de forma autónoma sus estados operativos.

1.1 Definición del problema

El trabajo consiste en implementar un modelo de agrupamiento (clustering) no supervisado mediante la red de Carpenter/Grossberg para descubrir y categorizar de forma autónoma los estados operativos de un servidor. A diferencia de un clasificador supervisado, la red no recibe etiquetas como normal o falla: agrupa el comportamiento del sistema solo a partir de la similitud entre sus perfiles de telemetría.
Una anomalía queda definida de forma operativa como la creación automática de un clúster nuevo y aislado: un comportamiento del servidor que la red no logra asimilar a ninguno de los prototipos aprendidos durante el régimen nominal (funcionamiento normal).

1.2 Justificación de la elección de la red

- Aprendizaje continuo: según Lippmann (1987), la red implementa el algoritmo del líder, en el que cada patrón nuevo se compara con los clústeres existentes y, si no se parece a ninguno, pasa a ser el ejemplar de un clúster propio. Esto le permite seguir aprendiendo en línea, algo importante en infraestructura informática, donde aparecen continuamente nuevos tipos de fallas o de carga que un modelo entrenado una sola vez no reconocería.
- No hay que fijar de antemano la cantidad de clústeres: la red genera dinámicamente tantas categorías como el comportamiento del servidor requiera, según la vigilancia elegida.
- Clasificador no paramétrico: no parte de suposiciones sobre la distribución estadística de la telemetría; solo compara patrones binarios.
- Estabilidad de lo aprendido: la red solo modifica un prototipo cuando el patrón nuevo supera el test de vigilancia. Un comportamiento distinto genera una categoría nueva en lugar de alterar las existentes.
- La binarización actúa como primer filtro de ruido: como la red trabaja con 0 y 1, cada métrica se convierte en una alerta que vale 1 solo cuando la medición es alta. Así se aíslan las fluctuaciones menores del servidor y se conservan los estados críticos.

2. Alcances y limitaciones

2.1 Alcances

- Agrupamiento no supervisado adaptativo: implementa la arquitectura ART-1 de Carpenter/Grossberg para agrupar perfiles de telemetría sin etiquetas de entrenamiento.
- Fiel al algoritmo de referencia: sigue los 6 pasos del Box 3 de Lippmann (1987). Esto incluye la inicialización de los nodos libres, es decir, los nodos de salida todavía no

comprometidos con ninguna categoría, que el artículo define con pesos t = 1 y b = 1/(1+N), donde N es la cantidad de variables. Las pruebas automáticas verifican que el resultado coincide con una implementación literal del Box 3.

- Aprendizaje en línea (fast learning): la red procesa los patrones uno por uno y en una sola pasada. Cuando un patrón no supera el test de vigilancia con ningún prototipo existente, crea dinámicamente una categoría nueva, que corresponde a un nuevo nodo en la capa de salida F2. Lippmann llama a este modo *fast learning* (aprendizaje rápido): el prototipo se actualiza por completo con cada patrón asignado.
- Formato estándar de entrada: procesa archivos CSV (archivos de texto donde cada fila es un registro y los valores se separan con comas) con encabezado y con cualquier cantidad de variables binarias (N) y de registros.
- Manual integrado: la opción -h o --help muestra el manual (man) del programa. Ante cualquier corrida infructuosa, el programa explica el error, muestra el manual y termina con código de salida 1, que es la forma estándar de avisarle al sistema operativo que hubo un error.
- Exportación de resultados: la opción -o guarda en un CSV el clúster asignado a cada registro.

2.2 Limitaciones

- Solo entradas binarias: ART-1 admite únicamente valores 0 o 1. Las variables continuas o categóricas deben binarizarse antes, es decir, convertirse a 0 y 1. El script preprocesamiento_art1.py hace esta conversión para la telemetría del trabajo.
- Vectores nulos: se rechazan los registros con todas las variables en 0. El test de vigilancia divide por la cantidad de unos del patrón ($\|X\|$); si no tiene ningún 1, esa división sería por cero.
- Sensibilidad al ruido: cada clúster guarda un patrón representativo llamado prototipo. Los prototipos se actualizan con una operación lógica AND: el prototipo conserva un 1 solo en las posiciones donde también hay un 1 en el patrón nuevo. Por eso, un bit en 0 por ruido (error de medición) borra para siempre ese bit del prototipo; a este efecto se lo llama erosión. Con vigilancia alta, el ruido además provoca categorías redundantes.
- Dependencia del orden: como en el algoritmo del líder, el resultado depende del orden en que se presentan las filas del CSV, porque el primer patrón de cada clúster define su prototipo inicial. Con otro orden pueden cambiar los tamaños de los clústeres e incluso la cantidad de categorías.
- Vigilancia menor que 1: el test de Lippmann es estricto: exige que el parecido sea mayor que ρ, no alcanza con igualarlo. Como el parecido máximo posible es 1, con ρ = 1 ningún patrón superaría el test. Por eso el programa exige $0 \leq \rho < 1$.
- Recursos finitos: con la opción -m se puede limitar la cantidad de nodos F2. Si se agotan, los patrones que no encajan quedan sin clasificar (clúster -1).

3. Requisitos e instalación

3.1 Requisitos

- Python 3.8 o superior.
- Sin dependencias externas: el programa usa solo la biblioteca estándar de Python.
- Sistema operativo: Windows, Linux o macOS.

3.2 Instalación paso a paso

1. Verificar la versión de Python (en Windows, usar python en lugar de python3):

```
python3 --version
```

2. Descargar el proyecto. Con Git:

git clone https://github.com/MauroMottet/TP-FINAL-IA.git
cd TP-FINAL-IA
Sin Git: en la página del repositorio, elegir Code → Download ZIP, descomprimir el archivo y abrir una terminal en esa carpeta.

3. Verificar que el programa funciona mostrando el manual integrado:

python3 CarGross.py --help

4. Opcional: ejecutar las pruebas automáticas. Si todo está bien, terminan con la palabra OK.

python3 -m unittest -v

3.3 Archivos del proyecto

Archivo	Descripción
CarGross.py	Programa principal: red de Carpenter/Grossberg
dataset_telemetria_base.csv	Dataset 1: régimen nominal (100 registros × 5 variables)
dataset_telemetria_estres.csv	Dataset 2: régimen de estrés (100 registros × 5 variables)
metadatos_variables.csv	Metadatos: métrica de origen, umbral y regla de cada variable binaria
preprocesamiento_art1.py	Genera los datasets binarios a partir de la telemetría cruda
machine_usage_days_1_to_8_grouped_300_seconds.csv	Telemetría cruda: 2243 registros tomados cada 300 segundos, con 5 métricas
correr_experimentos.py	Corridas del informe; guarda los resultados en la carpeta resultados/
test_cargross.py	Pruebas automáticas (16 pruebas)
README.md, requirements.txt	Instrucciones rápidas y lista de dependencias (vacía)

4. Modo de uso

4.1 Sintaxis

python3 CarGross.py -d <archivo.csv> [-p RHO] [-m MAX_NODOS] [-o SALIDA.csv] [-v]

Las opciones entre corchetes son optativas; si no se indican, se usa el valor por defecto.

Opción	Descripción	Por defecto
-d, --dataset	Ruta del CSV de entrada	obligatoria
-p, --rho	Vigilancia ρ: mayor o igual que 0 y menor que 1. Más alta = categorías más estrictas y más numerosas	0.5
-m, --max-nodos	Cantidad M de nodos de salida F2 disponibles	uno por registro
-o, --output	CSV donde se guarda el clúster asignado a cada registro	no se guarda

Opción	Descripción	Por defecto
-v, --verbose	Muestra paso a paso a qué clúster se asigna cada patrón	desactivado
-h, --help	Muestra el manual integrado	

4.2 Formato del archivo de entrada

- Archivo de texto CSV separado por comas (,) y codificado en UTF-8, que es la codificación estándar para letras con tilde. Se acepta el formato "CSV UTF-8" de Excel.
- La primera fila es el encabezado, con el nombre de cada variable.
- Cada fila siguiente es un registro: tantos valores como columnas del encabezado, todos 0 o 1, y al menos un 1.
- Las filas completamente vacías se ignoran.
-

Ejemplo mínimo:
cpu_alta,ram_alta,disco_saturado,red_in_alta,red_out_alta
1,0,0,0,0
0,0,1,0,0
0,1,0,1,1

4.3 Archivo de salida (opción -o)

CSV con dos columnas. fila es el número de registro, contando desde 1 a partir de la fila siguiente al encabezado. cluster es el número de nodo F2 asignado; vale -1 si el registro quedó sin clasificar porque se agotaron los nodos.

fila,cluster
1,0
2,1
3,0

4.4 Archivos de metadatos

CarGross.py no necesita archivos de metadatos para funcionar. El archivo metadatos_variables.csv es documentación: indica, para cada variable binaria de los datasets entregados, la métrica cruda de la que proviene, el umbral usado y la regla de binarización. Sus columnas son variable_binaria, columna_fuente, umbral_p90, regla y descripcion.

4.5 Manual integrado (salida de --help)

usage: CarGross.py [-h] -d DATASET [-p RHO] [-m MAX_NODOS] [-o OUTPUT] [-v]

==========
RED DE CARPENTER/GROSSBERG (ART-1) - CLASIFICADOR NO SUPERVISADO
==========
Este script implementa el algoritmo secuencial del líder basado en la red de Carpenter/Grossberg para clustering no supervisado sobre patrones estrictamente binarios.

REGLAS DE OPERACIÓN:
1. El archivo de entrada debe ser un CSV con valores 0 o 1.
2. La primera fila del CSV es el encabezado (nombres de las variables) y

todas las filas deben tener la misma cantidad de columnas, separadas por coma.
3. El parámetro de vigilancia (rho) debe estar en el intervalo [0.0, 1.0).
   El test de Lippmann es estricto (||T*X||/||X|| > rho): con rho = 1 ningún
   patrón superaría el test y cada fila quedaría sin clasificar.
4. No se admiten vectores completamente nulos (suma = 0) para evitar
   indeterminación matemática en el test de vigilancia (división por cero).
5. La red dispone de M nodos de salida (F2). Si se agotan, los patrones que no
   encajan en ninguna categoría se informan como "sin clasificar".

options:
  -h, --help   show this help message and exit
  -d DATASET, --dataset DATASET
                        Ruta al archivo CSV de entrada con variables binarias.
  -p RHO, --rho RHO   Umbral de vigilancia rho en [0.0, 1.0). Por defecto:
                        0.5.
  -m MAX_NODOS, --max-nodos MAX_NODOS
                        Cantidad M de nodos de salida F2. Por defecto: uno por
                        patrón (la red nunca se queda sin nodos).
  -o OUTPUT, --output OUTPUT
                        Ruta de un CSV donde guardar la asignación fila ->
                        clúster.
  -v, --verbose   Muestra la asignación paso a paso de cada vector.

EJEMPLOS DE USO:
  python3 CarGross.py -d dataset_telemetria_base.csv -p 0.6
  python3 CarGross.py --dataset dataset_telemetria_estres.csv --rho 0.8 --verbose
  python3 CarGross.py -d dataset_telemetria_base.csv -p 0.5 -o asignaciones.csv

SALIDA CSV (-o): columnas 'fila,cluster'. 'fila' es el número de registro de
datos (1 = primera fila después del encabezado) y 'cluster' es el nodo F2
asignado (-1 = sin clasificar por falta de nodos libres).

5. Test demo

5.1 Dataset 1: régimen nominal

Ejecutar en la carpeta del proyecto:
python3 CarGross.py -d dataset_telemetria_base.csv -p 0.5
Salida esperada:

```
============================================================
 RED NEURONAL DE CARPENTER/GROSSBERG - SESIÓN DE APRENDIZAJE
============================================================
 Dataset Fuente     : dataset_telemetria_base.csv
 Patrones Aplicados : 100
 Nodos de Entrada F1: 5 (cpu_alta, ram_alta, disco_saturado, red_in_alta, red_out_alta)
 Nivel de Vigilancia: 0.5
============================================================

------------------------------------------------------------
 RESUMEN DE LA TOPOLOGÍA RESULTANTE EN LA CAPA DE SALIDA (F2)
------------------------------------------------------------
 Total de Categorías Estabilizadas (M): 8

 Nodo F2 [00] ->  23 instancias ( 23.0%) | Vector Prototipo Top-Down: [0, 0, 1, 0, 0]
```

```
 Nodo F2 [01] ->  21 instancias ( 21.0%) | Vector Prototipo Top-Down: [1, 0, 0, 0, 0]
 Nodo F2 [02] ->  18 instancias ( 18.0%) | Vector Prototipo Top-Down: [1, 0, 1, 0, 0]
 Nodo F2 [03] ->  12 instancias ( 12.0%) | Vector Prototipo Top-Down: [0, 1, 0, 0, 0]
 Nodo F2 [04] ->   3 instancias (  3.0%) | Vector Prototipo Top-Down: [1, 1, 0, 0, 0]
 Nodo F2 [05] ->   1 instancias (  1.0%) | Vector Prototipo Top-Down: [0, 1, 1, 0, 0]
 Nodo F2 [06] ->   1 instancias (  1.0%) | Vector Prototipo Top-Down: [0, 0, 1, 0, 1]
 Nodo F2 [07] ->  21 instancias ( 21.0%) | Vector Prototipo Top-Down: [0, 0, 0, 1, 1]
============================================================
```

Cómo leer la salida: la capa de entrada F1 recibe 100 patrones de 5 componentes. Cada Nodo F2 es una categoría formada por la red; para cada una se indica cuántas instancias reunió y su prototipo top-down, que es el patrón de 0 y 1 que la representa y que se guarda en los pesos descendentes (de la capa F2 a la F1).
Interpretación: con ρ = 0.5 la red forma M = 8 categorías, una por cada combinación de alertas presente en el régimen nominal. Los estados dominantes son disco alto aislado (23 %), CPU alta aislada (21 %), tráfico de red alto en ambos sentidos (21 %) y CPU con disco (18 %). Las combinaciones raras (entre 1 % y 3 %) quedan aisladas en nodos propios y son candidatas a revisión.

5.2 Dataset 2: régimen de estrés

python3 CarGross.py -d dataset_telemetria_estres.csv -p 0.5
Salida esperada:

```
============================================================
 RED NEURONAL DE CARPENTER/GROSSBERG - SESIÓN DE APRENDIZAJE
============================================================
 Dataset Fuente     : dataset_telemetria_estres.csv
 Patrones Aplicados : 100
 Nodos de Entrada F1: 5 (cpu_alta, ram_alta, disco_saturado, red_in_alta, red_out_alta)
 Nivel de Vigilancia: 0.5
============================================================


------------------------------------------------------------
 RESUMEN DE LA TOPOLOGÍA RESULTANTE EN LA CAPA DE SALIDA (F2)
------------------------------------------------------------
 Total de Categorías Estabilizadas (M): 4

 Nodo F2 [00] ->  19 instancias ( 19.0%) | Vector Prototipo Top-Down: [1, 1, 1, 0, 0]
 Nodo F2 [01] ->  12 instancias ( 12.0%) | Vector Prototipo Top-Down: [0, 0, 0, 1, 1]
 Nodo F2 [02] ->  57 instancias ( 57.0%) | Vector Prototipo Top-Down: [0, 1, 0, 1, 1]
 Nodo F2 [03] ->  12 instancias ( 12.0%) | Vector Prototipo Top-Down: [1, 0, 1, 1, 1]
============================================================
```

Interpretación: con la misma vigilancia, la red forma solo M = 4 categorías, aunque el dataset tiene 8 combinaciones distintas. El estado dominante es memoria y red altas (57 %). El nodo 01 muestra el efecto del AND: su prototipo [0, 0, 0, 1, 1] no aparece tal cual en el dataset, porque todos sus registros tienen 3 alertas o más. Es la intersección de los patrones que absorbió: la red conservó el rasgo común (tráfico de red alto) y descartó el resto.

5.3 Otras pruebas sugeridas

- Ver la asignación paso a paso: python3 CarGross.py -d dataset_telemetria_base.csv -p 0.5 -v
- Probar distintas vigilancias, por ejemplo -p 0.3 y -p 0.9, y comparar la cantidad de categorías.
- Limitar los nodos F2: python3 CarGross.py -d dataset_telemetria_estres.csv -p 0.9 -m 3

- Ejecutar todas las corridas del informe: python3 correr_experimentos.py

6. Datasets entregados

Ambos datasets se generan con preprocesamiento_art1.py a partir de la telemetría cruda de un servidor: 2243 registros tomados cada 300 segundos (5 minutos) durante 8 días. Cada variable binaria vale 1 cuando la métrica supera su percentil 90, es decir, el valor que solo supera el 10 % más alto de las mediciones de esos 8 días:

Variable binaria	Métrica cruda	Umbral (P90)
cpu_alta	cpu_util_percent (uso de CPU, %)	53.51
ram_alta	mem_util_percent (uso de memoria, %)	91.15
disco_saturado	disk_io_percent (uso de disco, %)	12.59
red_in_alta	net_in (tráfico de red entrante)	46.11
red_out_alta	net_out (tráfico de red saliente)	36.61

- Los registros sin ninguna alerta (1634, el 72.85 %) se descartan porque el vector nulo no puede procesarse.
- Dataset 1, régimen nominal (dataset_telemetria_base.csv): 100 registros con 1 o 2 alertas simultáneas, elegidos al azar entre los 463 disponibles y ordenados en el tiempo. Se usa una semilla fija (42), un número que hace que la elección al azar sea siempre la misma y el dataset se pueda regenerar idéntico. Las 5 variables toman ambos valores y hay 8 combinaciones distintas.
- Dataset 2, régimen de estrés (dataset_telemetria_estres.csv): los primeros 100 registros con 3 o más alertas simultáneas. Hay 8 combinaciones distintas.
- Los dos datasets no comparten registros. Para regenerarlos se ejecuta python3 preprocesamiento_art1.py; el resultado es idéntico byte a byte.

7. Mensajes de error

En todos los casos el programa muestra el mensaje, a continuación el manual integrado, y termina con código de salida 1.

Situación	Mensaje	Solución
Se ejecuta sin argumentos	Muestra el manual	Indicar el dataset con -d
El archivo no existe	El archivo '…' no existe.	Revisar la ruta y la carpeta actual
Valor distinto de 0 o 1	Valor no binario detectado en fila F, col C	Binarizar los datos antes (ver preprocesamiento_art1.py)
Registro con todos 0	Fila F contiene un vector nulo	Eliminar esos registros
Fila con otra cantidad de columnas	Fila F tiene X columnas, pero el encabezado define Y	Corregir la fila o el encabezado
CSV separado por punto y coma	El archivo parece usar ';' como separador	Guardar como CSV separado por comas
Archivo vacío o sin registros	El archivo CSV está completamente vacío / No se encontraron filas	Revisar el contenido del archivo

Situación	Mensaje	Solución
Archivo que no es texto	El archivo no es texto UTF-8	Verificar que sea un CSV
ρ fuera de rango	Debe cumplir 0.0 <= rho < 1.0	Usar un valor mayor o igual que 0 y menor que 1 (por ejemplo, 0.99)
-m menor que 1	La cantidad de nodos F2 debe ser al menos 1	Usar un entero positivo
No se puede escribir la salida	No se pudo escribir '…'	Revisar la ruta y los permisos de -o

8. Preguntas frecuentes (FAQ)

¿Por qué el programa rechaza los vectores con todos los valores en 0?

Porque el test de vigilancia calcula qué fracción de los unos del patrón también está en el prototipo, y para eso divide por la cantidad de unos del patrón ($\|X\|$). Si el vector es nulo, la división es por cero y el resultado queda indeterminado. Esos registros deben filtrarse en el preprocesamiento; en la telemetría de este trabajo representan los períodos sin ninguna alerta.

¿Cómo influye la vigilancia ρ en la detección de anomalías?

Con ρ bajo (por ejemplo 0.3), la red es tolerante y fusiona patrones heterogéneos en pocas categorías amplias, con el riesgo de camuflar anomalías dentro del comportamiento normal. Con ρ alto (por ejemplo 0.9), exige coincidencias casi exactas: pequeñas variaciones o ruido disparan el reset (la red descarta el nodo candidato) y se crean muchas categorías. En el informe se muestra que, al presentar el régimen nominal y luego el de estrés, con $\rho \geq 0.7$ ningún patrón de estrés es absorbido por una categoría nominal, mientras que con $\rho \leq 0.3$ se absorben 96 de cada 100.

¿En qué se diferencia de K-Means o Fuzzy C-Means?

En K-Means y Fuzzy C-Means la cantidad de clústeres (K o c) se fija de antemano, los datos se recorren muchas veces (épocas) hasta que los centroides, es decir, los centros de los clústeres, convergen, y se aceptan valores continuos. La red de Carpenter/Grossberg no necesita conocer la cantidad de clústeres, aprende en una sola pasada y en línea (puede seguir incorporando patrones nuevos sin olvidar los anteriores) y, en su versión ART-1, solo procesa entradas binarias.

¿Por qué no se puede usar ρ = 1?

El test de Lippmann exige que la coincidencia sea estrictamente mayor que ρ. La coincidencia máxima posible es 1, así que con ρ = 1 ningún patrón superaría el test, ni siquiera uno idéntico a un prototipo. Para exigir coincidencia exacta alcanza con un valor como 0.99.

¿Por qué obtengo otro resultado si cambio el orden de las filas?

Porque la red aprende en línea: el primer patrón de cada clúster define el prototipo inicial y el AND lo va recortando con los siguientes. Es una propiedad del algoritmo del líder, no un error. En el informe se cuantifica este efecto con 20 órdenes aleatorios.

¿Cómo uso la red con mis propios datos?

Hay que binarizar cada variable (por ejemplo, comparándola con un umbral), eliminar los vectores nulos y guardar el resultado como CSV separado por comas, con encabezado. El script preprocesamiento_art1.py sirve de ejemplo.

REFERENCIAS

[1] Lippmann, R. P. (1987). An Introduction to Computing with Neural Nets. IEEE ASSP Magazine, 4(2), 4–22. (Modelo de Carpenter/Grossberg, algoritmo del líder y Box 3.)

[2] Bezdek, J. C. (1981). Pattern Recognition with Fuzzy Objective Function Algorithms. Plenum Press. (Fuzzy C-Means, usado en la comparación de la FAQ.)